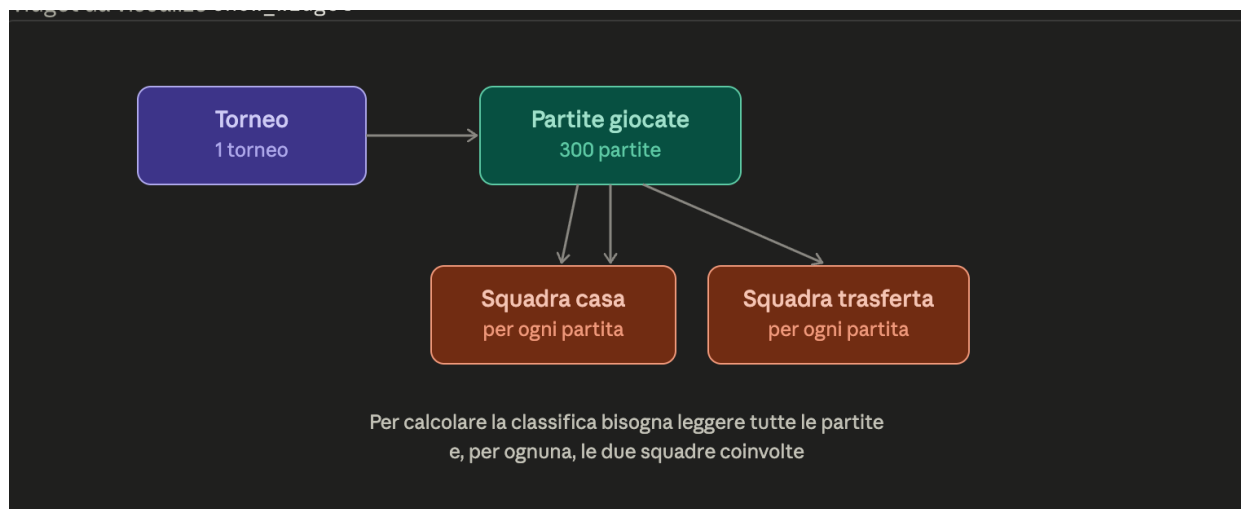


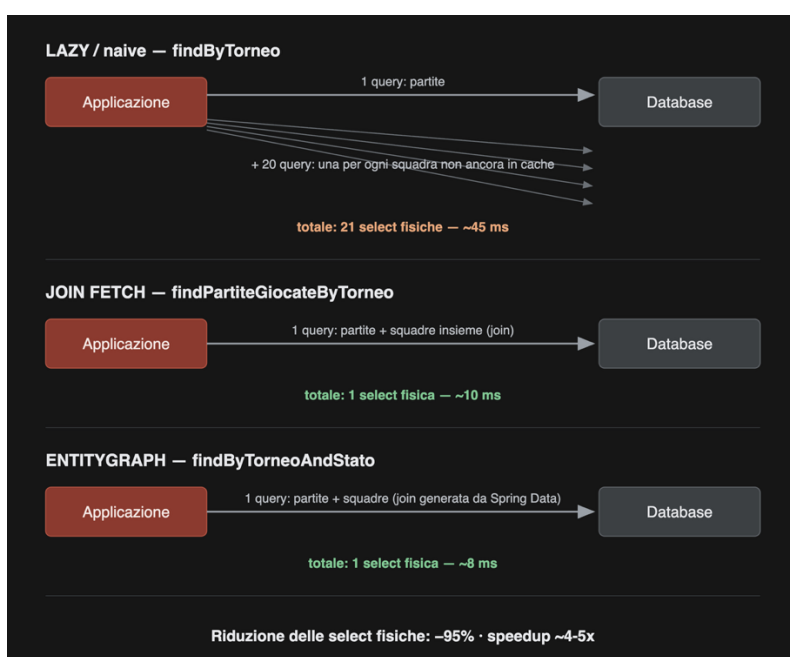
# ANALISI SPERIMENTALE

## Cos'è il test

Ho fatto un test per confrontare due modi diversi di leggere i dati dal database, per vedere qual è più veloce. Il caso che ho scelto è il calcolo della classifica di un torneo: bisogna leggere tutte le partite giocate e, per ognuna, le due squadre che hanno giocato.



Il file che utilizzo per il test è `PerformanceExperimentTest.java`. È un test di integrazione Spring Boot (`@SpringBootTest`) che confronta due strategie di fetch JPA/Hibernate sullo stesso caso d'uso.



## Il problema (metodo 1 - LAZY)

Nel primo modo, il programma prima prende tutte le partite con una query. Poi, per ogni partita, deve fare una query in più per prendere le squadre, perché non le ha ancora caricate. Con 300 partite questo crea tante query in più. Questo problema si chiama "N+1".

## La soluzione (metodo 2 - JOIN FETCH)

Nel secondo modo ho usato una query che prende partite e squadre tutte insieme, in un solo colpo. Così non servono query in più dopo.

## L'alternativa (metodo 3 - EntityGraph)

Il terzo metodo usa `@EntityGraph`: invece di scrivere la query a mano, si dice a Spring Data quali associazioni caricare subito (`squadraDiCasa` e `squadraDiTrasferta`) e la query la genera lui, sempre con una join. Il risultato è lo stesso del JOIN FETCH (una sola SELECT), ma in modo dichiarativo.

**Perché accade questo?** Succede perché le associazioni sono dichiarate LAZY: Hibernate al posto delle squadre mette dei "segnaposto" (proxy) e carica quella vera solo quando viene chiesta, una alla volta, con una query ciascuna. JOIN FETCH ed EntityGraph invece le fanno arrivare subito, tutte insieme nella stessa query delle partite.

## Come ho testato

Ho creato dati finti: 20 squadre e 300 partite giocate. Poi ho eseguito i tre metodi che stanno in `PartitaRepository.java` e ho contato quante query venivano fatte al database e quanto tempo ci mettevano.

## Conclusione

LAZY è una buona scelta come default, perché evita di caricare dati che non servono. Però quando si sa già che serviranno anche i dati collegati (come le squadre per la classifica), conviene usare JOIN FETCH o EntityGraph: si evita il problema N+1 e il programma diventa molto più veloce. Nel progetto infatti la classifica usa il metodo con JOIN FETCH.